

M2003 Series CMSIS BSP Guide

Directory Introduction for 32-bit NuMicro® Family

Directory Information

Please first extract the “M2003_Series_BSP_V3.01.000.zip” file and confirm the contents of this BSP folder.

M2003xC_M2003xD_M2003xE_M2003xG_Series	BSP for M2003xC_M2003xD_M2003xE_M2003xG_Series Related products include: M2003FC1AE, M2003XC1AE, M2003ED4AE, M2003TD4AE, M2003LD4AE, M2003TE4AE, M2003LE4AE, M2003LG6AE, M2003SG6AE, M2003RG6AE, M2003R1G6AE 256KB Flash APROM shared with Data flash, 4KB LDROM, 1KB SPROM, 36KB SRAM or 128KB/64KB Flash APROM shared with Data flash, 4KB LDROM, 1KB SPROM, 24KB SRAM In LQFP64, LQFP48, QFN33, LQFP32 & TSSOP28 package
M2003xI_M2003xJ_Series	BSP for M2003xI_M2003xJ_Series Related products include: M2003SI7AE, M2003VI7AE, M2003SJ7AE, M2003VJ7AE 1MB/512KB Flash APROM shared with Data flash, 4KB LDROM, 48KB SRAM In LQFP100 & LQFP64 package

Each of the folders listed above contains the following content:

Document	Driver reference guide and revision history.
Library	Driver header and source files.
SampleCode	Driver sample code.

The information described in this document is the exclusive intellectual property of Nuvoton Technology Corporation and shall not be reproduced without permission from Nuvoton.

Nuvoton is providing this document only for reference purposes of NuMicro microcontroller based system design. Nuvoton assumes no responsibility for errors or omissions.

All data and specifications are subject to change without notice.

For additional information or questions, please contact: Nuvoton Technology Corporation.

www.nuvoton.com

TABLE OF CONTENTS

1	DOCUMENT.....	4
2	LIBRARY	5
3	SAMPLECODE	6
4	SAMPLECODE\ISP	7
5	SAMPLECODE\POWERMANAGEMENT	8
6	SAMPLECODE\STDDRIVER.....	9
	System Manager (SYS)	9
	Clock Controller (CLK).....	9
	Flash Memory Controller (FMC)	9
	General Purpose I/O (GPIO).....	10
	Timer Controller (TIMER).....	10
	Watchdog Timer (WDT)	11
	Window Watchdog Timer (WWDT)	11
	PDMA Controller (PDMA)	11
	Pulse Width Modulation Controller (PWM)	11
	UART Interface Controller (UART).....	12
	I ² C Serial Interface Controller (I ² C)	13
	Quad Serial Peripheral Interface (QSPI)	13
	Universal Serial Control Interface Controller – UART Mode (USCI-UART)	14
	Universal Serial Control Interface Controller – SPI Mode (USCI-SPI)	14
	Universal Serial Control Interface Controller – I ² C Mode (USCI-I2C)	15
	Controller Area Network (CAN).....	15
	CRC Controller (CRC)	16
	Analog-to-Digital Converter (ADC)	16
	Enhanced Input Capture Timer (ECAP)	17
7	SAMPLE CODE COMPATIBILITY LIST	18

1 Document

CMSIS.html	Document of CMSIS version 6.1.0.
NuMicro M2003 Driver Reference Guide.chm	This document describes the usage of drivers in M2003 BSP.
NuMicro M2003 Series CMSIS BSP Revision History.pdf	This document shows the revision history of M2003 BSP.

2 Library

CMSIS	Cortex® Microcontroller Software Interface Standard (CMSIS) V6.1.0 definitions by Arm® Corp.
Device	CMSIS compliant device header file.
StdDriver	All peripheral driver header and source files.

3 SampleCode

Hard_Fault_Sample	Show hard fault information when hard fault happened. The hard fault handler shows some information included program counter, which is the address where the processor was executing when the hard fault occurs. The listing file (or map file) can show what function and instruction that was. It also shows the Link Register (LR), which contains the return address of the last function call. It can show the status where CPU comes from to get to this point.
ISP	Sample codes for In-System-Programming.
PowerManagement	Sample codes for power management.
Semihost	Show how to print and get character through IDE console window.
StdDriver	Sample code to demonstrate the usage of M2003 series MCU peripheral driver APIs.
Template	A project template for M2003 series MCU.

4 SampleCode\ISP

ISP_CAN	In-System-Programming Sample code through CAN interface.
ISP_I2C	In-System-Programming Sample code through I ² C interface.
ISP_QSPI	In-System-Programming Sample code through QSPI interface.
ISP_RS485	In-System-Programming Sample code through RS485 interface.
ISP_UART	In-System-Programming Sample code through UART interface.

5 SampleCode\PowerManagement

SYS_PowerDown_MinCurrent

Demonstrate how to minimize power consumption when entering power down mode.

6 SampleCode\StdDriver

System Manager (SYS)

SYS_BODWakeup	Demonstrate how to wake up system from Power-down mode by brown-out detector interrupt.
----------------------	---

Clock Controller (CLK)

CLK_ClockDetector	Demonstrate the usage of clock fail detector and clock frequency range detector function.
--------------------------	---

Flash Memory Controller (FMC)

FMC_APROM_EEPROM_Emu_WriteArray	EEPROM emulation sample using APROM via FMC ISP interface.
FMC_CRC32	Demonstrate how to use FMC CRC32 ISP command to calculate the CRC32 checksum of APROM and LDROM.
FMC_ExecInSRAM	Implement a code and execute it in SRAM to program embedded Flash.
FMC_IAP	Demonstrate FMC IAP boot mode and show how to use vector remap function. LDROM image was embedded in APROM image and be programmed to LDROM Flash at run-time. This sample also shows how to branch between APROM and LDROM.
FMC_MultiBoot	Implement a multi-boot system to boot from different applications in APROM or LDROM by VECMAP.
FMC_ReadAllOne	Demonstrate how to use FMC Read-All-One ISP command to verify APROM or LDROM pages are all 0xFFFFFFFF or not.
FMC_RW	Show FMC read Flash IDs, erase, read, and write functions.
FMC_SPROM_EEPROM_Emu_WriteArray	EEPROM emulation sample using SPROM via FMC ISP interface.

General Purpose I/O (GPIO)

GPIO_EINTAndDebounce	Show the usage of GPIO external interrupt function and de-bounce function.
GPIO_INT	Show the usage of GPIO interrupt function.
GPIO_OutputInput	Show how to set GPIO pin mode and use pin data input and output control.
GPIO_PowerDown	Show how to wake up system from Power-down mode by GPIO interrupt.

Timer Controller (TIMER)

TIMER_CaptureCounter	Show how to use the Timer capture function to capture Timer counter value.
TIMER_Delay	Demonstrate the usage of TIMER_Delay API to generate a 1 second delay.
TIMER_EventCounter	Use TM0 pin to demonstrate Timer event counter function.
TIMER_FreeCountingMode	Use the timer TM0_EXT pin to demonstrate timer free counting mode function. And displays the measured input frequency to UART console.
TIMER_InterTimerTriggerMode	Use the timer TM0 pin to demonstrate inter timer trigger mode function. Also display the measured input frequency to UART console.
TIMER_Periodic	Use the Timer periodic mode to generate Timer interrupt every 1 second.
TIMER_PeriodicINT	Implement Timer counting in periodic mode.
TIMER_SW_RTC	This sample code performs how to use software to simulate RTC. In power down mode, using the Timer to wake-up the MCU and add RTC value per second.
TIMER_TimeoutWakeup	Use timer to wake up system from Power-down mode periodically.

TIMER_ToggleOut	Demonstrate the Timer0 toggle out function on TM0 pin.
------------------------	--

Watchdog Timer (WDT)

WDT_TimeoutWakeupAndReset	Implement WDT time-out interrupt event to wake up system and generate time-out reset system event while WDT time-out reset delay period expired.
----------------------------------	--

Window Watchdog Timer (WWDT)

WWDT_ReloadCounter	Show how to reload the WWDT counter value.
---------------------------	--

PDMA Controller (PDMA)

PDMA_BasicMode	Use PDMA channel 1 to transfer data from memory to memory.
PDMA_ScatterGather	Use PDMA channel 1 to transfer data from memory to memory by scatter-gather mode.
PDMA_ScatterGather_PingPongBuffer	Use PDMA to implement Ping-Pong buffer by scatter-gather mode (memory to memory).
PDMA_TimeOut	Demonstrate PDMA channel 1 get/clear timeout flag with UART1.

Pulse Width Modulation Controller (PWM)

PWM_240KHz_SwitchDuty	Demonstrate how to set PWM0 channel 0 output 240 kHz waveform and switch duty in each 0.5%.
PWM_Brake	Demonstrate how to use PWM brake function.
PWM_Capture	Capture the PWM Channel 2 waveform by PWM Channel 0.
PWM_DeadTime	Demonstrate how to use PWM Dead Time function.
PWM_DoubleBuffer	Change duty cycle and period of output waveform by PWM double buffer function.

PWM_OutputWaveform	Demonstrate how to use PWM counter output waveform.
PWM_PDMA_Capture	Capture the PWM0 Channel 0 waveform by PWM0 Channel 2, and use PDMA to transfer captured data.
PWM_PDMA_Capture_1MHzSignal	Capture the PWM0 Channel 0 waveform by PWM0 Channel 2, and use PDMA to transfer captured data. The frequency of PWM Channel 0 is 1 MHz, which is used to test the maximum input frequency for PWM Capture function.
PWM_SwitchDuty	Change duty cycle of output waveform by configured period.
PWM_SyncStart	Demonstrate how to use PWM counter synchronous start function.

UART Interface Controller (UART)

UART_AutoBaudRate	Show how to use auto baud rate detection function.
UART_AutoFlow	Transmit and receive data using auto flow control.
UART_IrDA	Transmit and receive UART data in UART IrDA mode.
UART_LIN	Transmit LIN header and response.
UART_PDMA	Demonstrate UART transmit and receive function with PDMA.
UART_RS485	Transmit and receive data in UART RS485 mode.
UART_SingleWire	Transmit and receive data in UART single-wire mode.
UART_TxRxFunction	Transmit and receive data from PC terminal through RS232 interface.
UART_Wakeup	Show how to wake up system from Power-down mode by UART interrupt.

I²C Serial Interface Controller (I²C)

I2C_Double_Buffer_Slave	Demonstrate how to set I ² C two-level buffer in Slave mode to receive 256 bytes data from a master. This sample code needs to work with I2C_MultiBytes_Master.
I2C_EEPROM	Show how to use I ² C interface to access EEPROM.
I2C_Master	Show how a master accesses a slave. This sample code needs to work with I2C_Slave.
I2C_MultiBytes_Master	Show how to set I ² C Multi bytes API Read and Write data to Slave. This sample code needs to work with I2C_Slave.
I2C_PDMA_TRX	Demonstrate I ² C PDMA mode. Need to connect I2C0 (master) and I2C1 (slave).
I2C_SMBus_Master	Show how to control SMBus interface and use SMBus protocol between Host and Slave. This sample code needs to work with I2C_SMBus_Slave.
I2C_SMBus_Slave	Show how to control SMBus interface and use SMBus protocol between Host and Slave. This sample code needs to work with I2C_SMBus_Master.
I2C_SingleByte_Master	Show how to use I ² C Single byte API Read and Write data to Slave. This sample code needs to work with I2C_Slave.
I2C_Slave	Demonstrate how to set I ² C in Slave mode to receive 256 bytes data from a master. This sample code needs to work with I2C_Master.
I2C_Wakeup_Slave	Show how to wake up MCU from Power-down mode via the I ² C interface. This sample code needs to work with I2C_Master.

Quad Serial Peripheral Interface (QSPI)

QSPI_DualMode_Flash	Access SPI Flash using QSPI dual mode.
QSPI_QuadMode_Flash	Access SPI Flash using QSPI quad mode.

QSPI_Slave3Wire	Configure QSPI0 as Slave 3 wire mode and demonstrate how to communicate with an off-chip SPI Master device with FIFO mode. This sample code needs to work with SPI_MasterFIFOmode sample code.
------------------------	--

Universal Serial Control Interface Controller – UART Mode (USCI-UART)

USCI_UART_AutoBaudRate	Show how to use auto baud rate detection function.
USCI_UART_Autoflow	Transmit and receive data using auto flow control.
USCI_UART_PDMA	This is a USCI_UART PDMA demo and need to connect USCI_UART Tx and Rx.
USCI_UART_RS485	Transmit and receive data in RS485 mode.
USCI_UART_TxRxFunction	Transmit and receive data from PC terminal through a RS232 interface.
USCI_UART_Wakeup	Show how to wake up system from Power-down mode by USCI interrupt in UART mode.

Universal Serial Control Interface Controller – SPI Mode (USCI-SPI)

USCI_SPI_Loopback	Implement USCI_SPI0 Master loop back transfer. This sample code needs to connect USCI_SPI0_MISO pin and USCI_SPI0_MOSI pin together. It will compare the received data with transmitted data.
USCI_SPI_MasterMode	Configure USCI_SPI0 as Master mode and demonstrate how to communicate with an off-chip SPI Slave device. This sample code needs to work with USCI_SPI_SlaveMode sample code.
USCI_SPI_PDMA_Loopback	Demonstrate USCI_SPI data transfer with PDMA. USCI_SPI0 will be configured as Master mode and USCI_SPI1 will be configured as Slave mode. Both TX PDMA function and RX PDMA function will be enabled.
USCI_SPI_SlaveMode	Configure USCI_SPI0 as Slave mode and demonstrate how to communicate with an off-chip SPI Master device. This sample code needs to work with USCI_SPI_MasterMode sample code.

Universal Serial Control Interface Controller – I²C Mode (USCI-I2C)

USCI_I2C_EEPROM	Demonstrate how to access EEPROM through a USCI_I2C interface.
USCI_I2C_Loopback	Show how a master accesses 7-bit address Slave (loopback).
USCI_I2C_Loopback_10bit	Show how a master accesses 10-bit address Slave (loopback).
USCI_I2C_Master	Demonstrate how a Master accesses Slave. This sample code needs to work with USCI_I2C_Slave sample code.
USCI_I2C_Master_10bit	Demonstrate how a Master uses 10-bit addressing access Slave. This sample code needs to work with USCI_I2C_Slave_10bit sample code.
USCI_I2C_MultiBytes_Master	Demonstrate how to use multi-bytes API to access slave. This sample code needs to work with USCI_I2C_Slave sample code.
USCI_I2C_SingleByte_Master	Demonstrate how to use single byte API to access slave. This sample code needs to work with USCI_I2C_Slave sample code.
USCI_I2C_Slave	Demonstrate how to set USCI_I2C in slave mode to receive the data from a Master. This sample code needs to work with USCI_I2C_Master sample code.
USCI_I2C_Slave_10bit	Demonstrate how to set USCI_I2C in 10-bit addressing slave mode to receive the data from a Master. This sample code needs to work with USCI_I2C_Master_10bit sample code.
USCI_I2C_Wakeup_Slave	Demonstrate how to set USCI_I2C to wake up MCU from Power-down mode. This sample code needs to work with USCI_I2C_Master sample code.

Controller Area Network (CAN)

CAN_BasicMode_Rx	Demonstrate CAN bus receive a message with basic mode. This sample code could work with
-------------------------	---

	CAN_BasicMode_Tx sample code.
CAN_BasicMode_Tx	Demonstrate CAN bus transmit a message with basic mode. This sample code could work with CAN_BasicMode_Rx sample code.
CAN_BasicMode_SilentMode	Demonstrate CAN Silent mode to listen to CAN bus communication test in Basic mode.
CAN_NormalMode_Rx	Demonstrate CAN bus receive a message with normal mode. This sample code could work with CAN_NormalMode_Tx sample code.
CAN_NormalMode_Tx	Demonstrate CAN bus transmit a message with normal mode. This sample code could work with CAN_NormalMode_Rx sample code.
CAN_NormalMode_SilentMode	Demonstrate CAN Silent mode to listen to CAN bus communication test in Normal mode.

CRC Controller (CRC)

CRC_CCITT	Implement CRC in CRC-CCITT mode and get the CRC checksum result.
CRC_CRC8	Implement CRC in CRC-8 mode and get the CRC checksum result.
CRC_CRC32_PDMA	Implement CRC in CRC-32 mode and get the CRC checksum result.
CRC_POLYNOMIAL	Demonstrate how to use polynomial mode and get the CRC checksum result.

Analog-to-Digital Converter (ADC)

ADC_ADINT_Trigger	Use ADINT interrupt to do the ADC Single-cycle scan conversion.
ADC_BandGap	Convert Band-gap and print conversion result.
ADC_BandGapCalculateAVDD	Demonstrate how to calculate analog voltage (AVdd) by

	using band-gap.
ADC_BurstMode	Perform A/D Conversion with ADC burst mode.
ADC_ContinuousScanMode	Perform A/D Conversion with ADC continuous scan mode.
ADC_PDMA_PWM_Trigger	Demonstrate how to trigger ADC by PWM and transfer conversion data by PDMA.
ADC_PWM_Trigger	Demonstrate how to trigger ADC by PWM.
ADC_ResultMonitor	Monitor the conversion result of channel 2 by the digital compare function.
ADC_SingleCycleScanMode	Perform A/D Conversion with ADC single cycle scan mode.
ADC_SingleMode	Perform A/D Conversion with ADC single mode.
ADC_STADC_Trigger	Show how to trigger ADC by STADC pin.
ADC_SwTrg_Trigger	Trigger ADC by writing ADC software trigger register.
ADC_TempSensor	Convert temperature sensor and print conversion result.
ADC_Timer_Trigger	Show how to trigger ADC by Timer.

Enhanced Input Capture Timer (ECAP)

ECAP_GetInputFreq	Show how to use ECAP interface to get input frequency.
--------------------------	--

7 Sample Code Compatibility List

Category Type Sample Code	M2003xC	M2003xD	M2003xE	M2003xG
ADC_ADINT_Trigger	√	√	√	√
ADC_BandGap	√	√	√	√
ADC_BandGapCalculateAVDD	√	√	√	√
ADC_BurstMode	√	√	√	√
ADC_ContinuousScanMode	√	√	√	√
ADC_PWM_Trigger	√	√	√	√
ADC_ResultMonitor	√	√	√	√
ADC_SingleCycleScanMode	√	√	√	√
ADC_SingleMode	√	√	√	√
ADC_STADC_Trigger	√	√	√	√
ADC_SwTrg_Trigger	√	√	√	√
ADC_Timer_Trigger	√	√	√	√
CAN_BasicMode_Rx	√		√	√
CAN_BasicMode_SilentMode	√		√	√
CAN_BasicMode_Tx	√		√	√
CAN_NormalMode_Rx	√		√	√
CAN_NormalMode_SilentMode	√		√	√
CAN_NormalMode_Tx	√		√	√
CLK_ClockDetector	√	√	√	√
CRC_CCITT				√
CRC_CRC32_PDMA				√
CRC_CRC8				√
ECAP_GetInputFreq	√	√	√	√

FMC_APROM_EEPROM_Emu_WriteArray	√	√	√	√
FMC_CRC32	√	√	√	√
FMC_ExecInSRAM	√	√	√	√
FMC_IAP	√	√	√	√
FMC_MultiBoot	√	√	√	√
FMC_ReadAllOne	√	√	√	√
FMC_RW	√	√	√	√
FMC_SPROM_EEPROM_Emu_WriteArray	√	√	√	√
GPIO_EINTAndDebounce	√	√	√	√
GPIO_INT	√	√	√	√
GPIO_OutputInput	√	√	√	√
GPIO_PowerDown	√	√	√	√
I2C_Double_Buffer_Slave	√	√	√	√
I2C_EEPROM	√	√	√	√
I2C_Master	√	√	√	√
I2C_MultiBytes_Master	√	√	√	√
I2C_PDMA_TRX	√	√	√	√
I2C_SingleByte_Master	√	√	√	√
I2C_Slave	√	√	√	√
I2C_SMBus_Master	√	√	√	√
I2C_SMBus_Slave	√	√	√	√
I2C_Wakeup_Slave	√	√	√	√
PDMA_BasicMode				√
PDMA_ScatterGather				√
PDMA_ScatterGather_PingPongBuffer				√
PDMA_TimeOut				√
PWM_240kHz_SwitchDuty	√	√	√	√

PWM_Brake	√	√	√	√
PWM_Capture	√	√	√	√
PWM_DeadTime	√	√	√	√
PWM_DoubleBuffer	√	√	√	√
PWM_OutputWaveform	√	√	√	√
PWM_PDMA_Capture	√	√	√	√
PWM_PDMA_Capture_1MHzSignal	√	√	√	√
PWM_SwitchDuty	√	√	√	√
PWM_SyncStart	√	√	√	√
QSPI_DualMode_Flash				√
QSPI_QuadMode_Flash				√
QSPI_Slave3Wire				√
SYS_BODWakeup	√	√	√	√
TIMER_CaptureCounter	√	√	√	√
TIMER_Delay	√	√	√	√
TIMER_EventCounter	√	√	√	√
TIMER_FreeCountingMode	√	√	√	√
TIMER_InterTimerTriggerMode	√	√	√	√
TIMER_Periodic	√	√	√	√
TIMER_PeriodicINT	√	√	√	√
TIMER_SW_RTC	√	√	√	√
TIMER_TimeoutWakeup	√	√	√	√
TIMER_ToggleOut	√	√	√	√
UART_AutoBaudRate	√	√	√	√
UART_AutoFlow	√	√	√	√
UART_IrDA	√	√	√	√
UART_LIN	√	√	√	√

UART_PDMA	√	√	√	√
UART_RS485	√	√	√	√
UART_SingleWire	√	√	√	√
UART_TxRxFunction	√	√	√	√
UART_Wakeup	√	√	√	√
USCI_I2C_EEPROM	√	√	√	√
USCI_I2C_Loopback	√	√	√	√
USCI_I2C_Loopback_10bit	√	√	√	√
USCI_I2C_Master	√	√	√	√
USCI_I2C_Master_10bit	√	√	√	√
USCI_I2C_MultiBytes_Master	√	√	√	√
USCI_I2C_SingleByte_Master	√	√	√	√
USCI_I2C_Slave	√	√	√	√
USCI_I2C_Slave_10bit	√	√	√	√
USCI_I2C_Wakeup_Slave	√	√	√	√
USCI_SPI_Loopback	√	√	√	√
USCI_SPI_MasterMode	√	√	√	√
USCI_SPI_PDMA_Loopback	√	√	√	√
USCI_SPI_SlaveMode	√	√	√	√
USCI_UART_AutoBaudRate	√	√	√	√
USCI_UART_AutoFlow	√	√	√	√
USCI_UART_PDMA	√	√	√	√
USCI_UART_RS485	√	√	√	√
USCI_UART_TxRxFunction	√	√	√	√
USCI_UART_Wakeup	√	√	√	√
WDT_TimeoutWakeupAndReset	√	√	√	√
WWDT_ReloadCounter	√	√	√	√

Important Notice

Nuvoton Products are neither intended nor warranted for usage in systems or equipment, any malfunction or failure of which may cause loss of human life, bodily injury or severe property damage. Such applications are deemed, “Insecure Usage”.

Insecure usage includes, but is not limited to: equipment for surgical implementation, atomic energy control instruments, airplane or spaceship instruments, the control or operation of dynamic, brake or safety systems designed for vehicular use, traffic signal instruments, all types of safety devices, and other applications intended to support or sustain life.

All Insecure Usage shall be made at customer’s risk, and in the event that third parties lay claims to Nuvoton as a result of customer’s Insecure Usage, customer shall indemnify the damages and liabilities thus incurred by Nuvoton.

*Please note that all data and specifications are subject to change without notice.
All the trademarks of products and companies mentioned in this datasheet belong to their respective owners.*